PROYEK TUGAS AKHIR SEMESTER MACHINE LEARNING

ANALISIS DETEKSI DAN PREDIKSI KECEPATAN MOBIL DARI PEMROSESAN VIDEO MENGGUNAKAN HAAR CASCADE CLASSIFIER

Diajukan Sebagai Tugas Akhir Semester Mata Kuliah Machine Learning

Jurusan Fisika Fakultas Sains Dan Teknologi

Oleh:

1. **Annisa Nurlaili Aulia Safitri (NIM. 1217030006)**
2. **Milah Nurlaila Apipah (NIM. 1217030020)**
3. **Shilviana Maula Afisa (NIM. 1217030037)**

JURUSAN FISIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI SUNAN GUNUNG DJATI BANDUNG

2024

BAB I

PENDAHULUAN

A. Latar Belakang

Analisis pergerakan objek dalam video merupakan salah satu topik utama dalam pengolahan citra digital dan *Computer Vision*. Kemampuan untuk mendeteksi objek, melacak pergerakannya, dan menganalisis parameter terkait, seperti kecepatan, memiliki berbagai aplikasi di banyak bidang, termasuk sains, olahraga, pengawasan, dan teknologi transportasi.

Secara tradisional, analisis pergerakan dilakukan secara manual, yang memerlukan waktu lama dan sering kali rentan terhadap kesalahan subjektif. Namun, dengan berkembangnya teknologi pengolahan citra, pendekatan berbasis algoritma kini memungkinkan proses ini dilakukan secara otomatis dan efisien. Salah satu metode populer yang digunakan untuk deteksi objek adalah *Haar Cascade Classifier*, yang dirancang untuk mengenali pola visual tertentu dalam video. Setelah objek terdeteksi, analisis pergerakan dilakukan dengan melacak perubahan posisi objek di setiap frame.

Dalam analisis ini, parameter utama yang dihitung adalah kecepatan. Kecepatan dihitung berdasarkan perpindahan posisi objek dalam satuan piksel yang dikonversi ke satuan fisik (misalnya meter), dengan mempertimbangkan waktu antar frame yang diperoleh dari kecepatan pemutaran video (*frames per second*). Data kecepatan kemudian divisualisasikan untuk memberikan wawasan tentang pola pergerakan objek secara dinamis.

Pendekatan ini bertujuan untuk menyediakan sistem yang dapat menganalisis pergerakan berbagai jenis objek secara real-time dan akurat. Implementasi metode ini diharapkan dapat diterapkan dalam berbagai konteks, seperti penelitian ilmiah untuk menganalisis gerak partikel, pengawasan untuk mendeteksi aktivitas mencurigakan, pengukuran kinerja dalam olahraga, atau simulasi fisik dalam pendidikan.

B. Tujuan Penelitian

1. Mengembangkan sistem berbasis Haar Cascade Classifier untuk mendeteksi kendaraan dalam sebuah video.

2. Memprediksi kecepatan kendaraan berdasarkan setiap frame pada video.

BAB II

DASAR TEORI

A. OpenCV dan Pengolahan Citra Digital

OpenCV (*Open Source Computer Vision Library*) adalah pustaka sumber terbuka yang menyediakan berbagai alat untuk pengolahan citra dan video, memungkinkan implementasi algoritma dalam berbagai bahasa pemrograman, termasuk Python, C++, dan Java. Pustaka ini memfasilitasi pengembangan aplikasi komputer visi dengan berbagai fungsi utama, seperti deteksi objek, pemrosesan citra, serta penerapan kecerdasan buatan dan pembelajaran mesin. Salah satu algoritma terkenal dalam OpenCV adalah *Haar Cascade Classifier*, yang diperkenalkan oleh Viola dan Jones (2001) untuk deteksi objek. Algoritma ini bekerja dengan menggunakan fitur *Haar* untuk mendeteksi perbedaan intensitas piksel dalam citra, memprosesnya melalui beberapa tingkat klasifikasi bertingkat, dan efektif untuk mendeteksi objek dengan posisi frontal atau hampir frontal dengan kecepatan tinggi. Selain itu, OpenCV menyediakan kemampuan untuk mengubah citra berwarna menjadi grayscale, yang menyederhanakan proses pemrosesan dan meningkatkan akurasi deteksi dengan menghilangkan informasi warna yang tidak relevan.

Implementasi *Haar Cascade* dalam OpenCV melibatkan proses preprocessing citra, seperti transformasi *grayscale*, diikuti dengan penerapan algoritma untuk mendeteksi objek, seperti wajah, mata, atau kendaraan. Aplikasi OpenCV dalam pengolahan citra digital meliputi sistem keamanan berbasis biometrik, analisis lalu lintas kendaraan, serta deteksi dan pengenalan wajah dalam aplikasi pengawasan. Meskipun OpenCV dan *Haar Cascade Classifier* memiliki banyak keunggulan, seperti efisiensi dan kecepatan dalam deteksi objek, teknologi ini juga memiliki keterbatasan, seperti ketergantungan pada posisi objek dan kondisi pencahayaan yang dapat mempengaruhi akurasi deteksi. Secara keseluruhan, OpenCV dan *Haar Cascade Classifier* tetap menjadi alat utama dalam pengembangan aplikasi komputer visi, meskipun perlu memperhatikan keterbatasan dalam situasi tertentu.

B. Haar Cascade Classifier

Haar Cascade Classifier adalah algoritma berbasis visi komputer yang digunakan untuk mendeteksi objek dalam citra atau video. Algoritma ini memanfaatkan fitur Haar-Like, yaitu perbedaan intensitas antara area terang dan gelap, untuk mengidentifikasi pola pada citra. Fitur-fitur ini diekstraksi menggunakan filter persegi panjang yang diterapkan pada berbagai wilayah citra. Proses selanjutnya melibatkan algoritma Adaboost untuk memilih fitur-fitur yang paling relevan dan menggabungkannya menjadi pengklasifikasi yang lebih kuat. Deteksi dilakukan melalui struktur kaskade, di mana setiap fitur dievaluasi secara berurutan. Wilayah yang tidak memenuhi kriteria langsung diabaikan, sehingga algoritma ini sangat efisien dan cocok untuk aplikasi waktu nyata.

Keunggulan Haar Cascade Classifier terletak pada efisiensi komputasi yang tinggi dan kemampuannya dalam mendeteksi objek secara real-time. Algoritma ini telah banyak diterapkan dalam berbagai bidang, seperti deteksi wajah dalam sistem keamanan, pemantauan lalu lintas untuk mendeteksi kendaraan, dan robotika untuk pengenalan objek. Berkat pendekatan berbasis fitur yang terstruktur, Haar Cascade Classifier memberikan solusi efektif dan praktis untuk kebutuhan deteksi objek di berbagai aplikasi pengolahan citra dan visi komputer.

C. Tracking dan Perhitungan Kecepatan Kendaraan

Tracking dan perhitungan kecepatan kendaraan adalah aplikasi penting dalam bidang pengolahan citra dan visi komputer. Dengan memanfaatkan algoritma seperti Haar Cascade Classifier dan teknik-teknik lain dari pustaka OpenCV, proses ini dapat dilakukan dengan menggabungkan deteksi objek, pelacakan, dan analisis pergerakan.

Haar Cascade Classifier digunakan untuk mendeteksi kendaraan dalam video atau gambar. Algoritma ini bekerja dengan melatih model berbasis gambar positif (dengan kendaraan) dan gambar negatif (tanpa kendaraan), menghasilkan kemampuan deteksi yang efisien. Setelah kendaraan terdeteksi, teknik pelacakan seperti optical flow dapat digunakan untuk memantau pergerakan kendaraan antar frame video. Dengan menganalisis perubahan posisi kendaraan, kecepatan dapat dihitung berdasarkan waktu yang dibutuhkan untuk berpindah jarak tertentu.

Implementasi sistem seperti ini biasanya melibatkan beberapa tahapan, termasuk preprocessing citra, seperti mengubah gambar menjadi grayscale untuk efisiensi pemrosesan.

Selain itu, deteksi awal objek dengan Haar Cascade memungkinkan sistem untuk mengenali kendaraan meski terdapat gangguan seperti variasi pencahayaan. Pengukuran kecepatan didasarkan pada data posisi kendaraan yang dikumpulkan dari frame video, dengan akurasi yang bergantung pada kualitas input dan kecepatan pemrosesan.

D. Visualisasi Data

Proses visualisasi data merupakan langkah penting dalam analisis untuk menyajikan informasi secara lebih mudah dipahami. Proses ini dimulai dengan pengumpulan data yang relevan untuk dianalisis. Setelah itu, data tersebut diproses melalui beberapa tahapan, seperti pembersihan untuk menghilangkan nilai yang tidak konsisten atau kosong, serta transformasi untuk menyesuaikan formatnya agar lebih mudah divisualisasikan.

Tahapan berikutnya adalah pemilihan jenis visualisasi yang sesuai dengan jenis data dan tujuan analisis, seperti diagram batang untuk perbandingan, diagram garis untuk tren, atau peta untuk data geografis. Setelah visualisasi dipilih, data akan diolah menggunakan perangkat lunak atau alat seperti Excel, Tableau, atau pustaka pemrograman seperti OpenCV di Python. Alat ini memungkinkan pengaturan elemen visual seperti skala, warna, dan anotasi untuk meningkatkan pemahaman.

Hasil visualisasi perlu diuji untuk memastikan akurasi representasi data dan kejelasan pesan yang disampaikan. Setelah selesai, visualisasi ini dapat digunakan untuk mendukung pengambilan keputusan, disajikan dalam laporan, atau dipublikasikan untuk konsumsi audiens yang lebih luas. Keseluruhan proses ini menekankan pentingnya integritas data, pemilihan desain visual yang tepat, dan komunikasi yang efektif untuk menghasilkan wawasan yang berharga.

BAB III

METODE PENELITIAN

3.1 Alat dan Bahan

No.	Alat dan Bahan	Jumlah
1.	Laptop	1 Buah
2.	Video Data	1 Video
3.	Website Google Colab	-
4.	Library Pemrograman	-

3.2 Prosedur Penelitian

Pada penelitian yang telah dilakukan, untuk mendeteksi pergerakan dan prediksi kecepatan mobil digunakan website Google Colab sebagai bahasa pemrograman yang digunakan. Video data yang akan dianalisis disimpan dalam folder dataset di Google Drive terlebih dahulu, lalu file define atau classifier disimpan dalam folder yang sama. Setelah dataset sudah selesai dibuat, selanjutnya website Google Colab dibuka dan digunakan perintah 'pip install opencv-python' untuk menginstal library Open CV (Open Source Computer Vision Library). Selanjutnya buat kode program untuk menganalisis video sehingga diperoleh hasil output berupa visualisasi plot grafik dan kecepatan objek.

Kode program yang digunakan pada sel pertama digunakan untuk membaca file video dan model deteksi Haar Cascade. Selanjutnya video frame demi frame didekode dan setiap frame dikonversi ke grayscale. Lalu, mobil dideteksi menggunakan Haar Cascade. Setelah itu, digambar kotak di sekitar mobil yang terdeteksi lalu hasil analisis ditampilkan di Google Colab menggunakan cv2_imshow.

Kode program yang digunakan pada sel kedua digunakan untuk membuka video dan Haar Cascade dimuat. Selanjutnya, frame dibaca dan mobil dideteksi dan posisi mobil dilacak antar frame. Lalu, kecepatan dihitung berdasarkan pergerakan. Setelah itu, hasil ditampilkan pada frame dan divisualisasikan dalam bentuk prediksi kecepatan.

BAB IV

DATA DAN PEMBAHASAN

4.1 Kode Program

Berikut adalah penjelasan kode program:

A. Kode Program Deteksi Benda

Pada pemrogram deteksi benda, digunakan library Haar Cascade Classifier yang diperoleh dari https://github.com/jitendrasb24/Car-Detection-OpenCV, begitu pun dengan kode programnya.

```
[ ] from google.colab.patches import cv2_imshow
    import cv2

    # Load video and classifier
    cap = cv2.VideoCapture('/content/dataset/carssss.mp4')
    car_cascade = cv2.CascadeClassifier('/content/dataset/haarcascade_cars.xml')

    if not cap.isOpened():
        print("Error: Cannot open video file.")
        exit()

    while True:
        ret, frame = cap.read()
        if not ret:
            print("End of video or cannot fetch the frame.")
            break

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        cars = car_cascade.detectMultiScale(gray, 1.1, 3)

        for (x, y, w, h) in cars:
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

        # Replace cv2.imshow with cv2_imshow
        cv2_imshow(frame)

        # Optional delay to mimic playback (reduce speed if needed)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()
```

1. Import Library

from google.colab.patches import cv2_imshow: Fungsi ini digunakan untuk menampilkan gambar atau video frame di Google Colab, karena cv2.imshow tidak dapat digunakan langsung di lingkungan Google Colab.

import cv2: Mengimpor pustaka OpenCV, yang digunakan untuk pengolahan gambar dan

video.

2. Load Video dan Haar Cascade Classifier

cv2.VideoCapture('/content/dataset/carssss.mp4'): Membuka file video bernama carssss.mp4 dari direktori /content/dataset/. Objek ini memungkinkan pembacaan frame dari video.

cv2.CascadeClassifier('/content/dataset/haarcascade_cars.xml'): Memuat model Haar Cascade yang telah dilatih sebelumnya untuk mendeteksi mobil. File XML ini berisi data fitur yang diperlukan untuk deteksi.

cap.isOpened(): Mengecek apakah file video berhasil dibuka. Jika tidak, program mencetak pesan error dan keluar.

3. Membaca Frame Secara Berulang

cap.read(): Membaca satu frame dari video. Fungsi ini mengembalikan:

- ret: Boolean yang menunjukkan apakah frame berhasil dibaca.
- frame: Frame video dalam bentuk array gambar.

if not ret:: Jika frame tidak dapat dibaca (misalnya, video sudah selesai), program mencetak pesan dan keluar dari loop.

4. Mengonversi ke Grayscale

cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY): Mengonversi frame berwarna (BGR) menjadi grayscale, karena Haar Cascade bekerja lebih baik pada gambar grayscale.

5. Deteksi Mobil

car_cascade.detectMultiScale(gray, 1.1, 3): Mendeteksi mobil dalam frame grayscale.

- gray: Frame dalam format grayscale.
- 1.1: Faktor skala, yang menentukan seberapa banyak ukuran gambar akan dikurangi setiap kali gambar diperiksa.
- 3: Jumlah minimum tetangga yang diperlukan untuk mempertahankan kandidat kotak

pembatas. Nilai lebih tinggi berarti deteksi lebih ketat.

6. Menandai Mobil yang Terdeteksi

for (x, y, w, h) in cars: Mengiterasi koordinat bounding box dari setiap mobil yang terdeteksi.

- x, y: Koordinat sudut kiri atas bounding box.
- w, h: Lebar dan tinggi bounding box.

cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2): Menggambar persegi panjang di sekitar mobil yang terdeteksi.

- (x, y), (x+w, y+h): Dua titik sudut persegi panjang.
- (0, 255, 0): Warna hijau.
- 2: Ketebalan garis persegi panjang.

7. Menambahkan Delay untuk Playback

cv2.waitKey(1): Menambahkan jeda selama 1 ms antara frame untuk mensimulasikan pemutaran video.

& 0xFF == ord('q'): Jika tombol "q" ditekan, program keluar dari loop.

8. Membersihkan Resource

cap.release(): Menutup file video dan melepaskan resource terkait.

cv2.destroyAllWindows(): Menutup semua jendela yang dibuat oleh OpenCV.

B. Kode Program Prediksi Kecepatan

```python
from google.colab.patches import cv2_imshow
import cv2
import numpy as np
import matplotlib.pyplot as plt
from collections import deque

# Load video and classifier
cap = cv2.VideoCapture('/content/dataset/carssss.mp4')
car_cascade = cv2.CascadeClassifier('/content/dataset/haarcascade_cars.xml')

if not cap.isOpened():
    print("Error: Cannot open video file.")
    exit()

# Parameters
fps = cap.get(cv2.CAP_PROP_FPS)  # Frames per second of the video
pixel_to_meter = 0.05  # Conversion factor: 1 pixel = 0.05 meters (adjust as needed)

# Variables for tracking
prev_centroids = {}
speed_data = {}
time_interval = 1 / fps
speed_history = deque(maxlen=50)

while True:
    ret, frame = cap.read()
    if not ret:
        print("End of video or cannot fetch the frame.")
        break
```

```python
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    cars = car_cascade.detectMultiScale(gray, 1.1, 3)

    current_centroids = {}

    for i, (x, y, w, h) in enumerate(cars):
        # Draw bounding box
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

        # Compute centroid
        cx, cy = x + w // 2, y + h // 2
        current_centroids[i] = (cx, cy)

        # Track vehicle and calculate speed
        if i in prev_centroids:
            prev_cx, prev_cy = prev_centroids[i]
            distance_pixels = np.sqrt((cx - prev_cx)**2 + (cy - prev_cy)**2)
            distance_meters = distance_pixels * pixel_to_meter
            speed = distance_meters / time_interval  # meters per second
            speed_data[i] = speed
            speed_history.append(speed)

            # Display speed on the frame
            cv2.putText(frame, f"{speed:.2f} m/s", (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2)

    # Update centroids
    prev_centroids = current_centroids

    # Show frame
    cv2_imshow(frame)
```

```
    # Update centroids
    prev_centroids = current_centroids

    # Show frame
    cv2_imshow(frame)

    # Plot speed graph
    plt.clf()
    plt.title("Vehicle Speed (m/s)")
    plt.xlabel("Frame")
    plt.ylabel("Speed")
    plt.plot(list(speed_history), label="Speed")
    plt.legend()
    plt.pause(0.001)

    # Break condition
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
plt.show()
```

1. Import Library

from google.colab.patches import cv2_imshow: Digunakan untuk menampilkan gambar atau video frame di Google Colab, karena fungsi bawaan OpenCV cv2.imshow() tidak kompatibel dengan lingkungan Colab.

import cv2: OpenCV digunakan untuk pengolahan gambar dan video.

import numpy as np: Numpy digunakan untuk operasi numerik seperti menghitung jarak piksel.

import matplotlib.pyplot as plt: Matplotlib digunakan untuk membuat grafik kecepatan kendaraan.

from collections import deque: Digunakan untuk menyimpan data kecepatan dengan kapasitas terbatas (history kecepatan).

2. Memuat Video dan Model Klasifikasi

cv2.VideoCapture: Membuka file video carssss.mp4 dari folder dataset.

cv2.CascadeClassifier: Memuat file model deteksi kendaraan berbasis Haar Cascade, haarcascade_cars.xml.

3. Parameter Variabel

fps: Mengambil informasi frame rate video.

pixel_to_meter: Faktor konversi dari piksel ke meter. Nilai ini harus disesuaikan dengan skala

kamera.

4. Proses Frame Video

prev_centroids: Menyimpan koordinat centroid kendaraan pada frame sebelumnya.

speed_data: Menyimpan data kecepatan setiap kendaraan.

time_interval: Interval waktu antar frame berdasarkan FPS.

speed_history: Menyimpan riwayat kecepatan kendaraan (maksimal 50 data).

5. Menggambar Kotak dan menghtiung Kecepatan

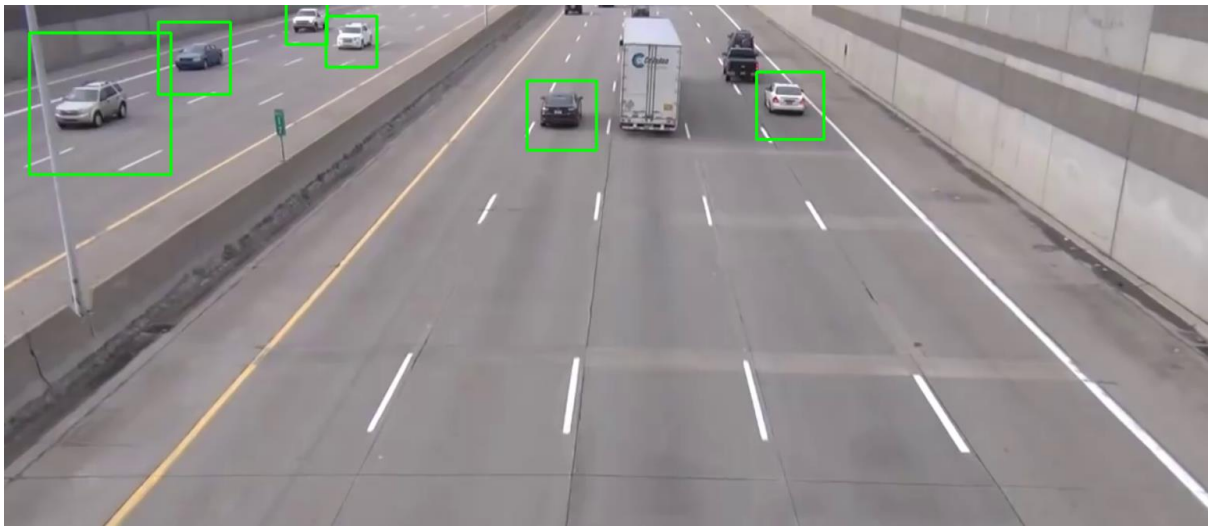
cv2.cvtColor: Mengonversi frame menjadi grayscale untuk deteksi objek.

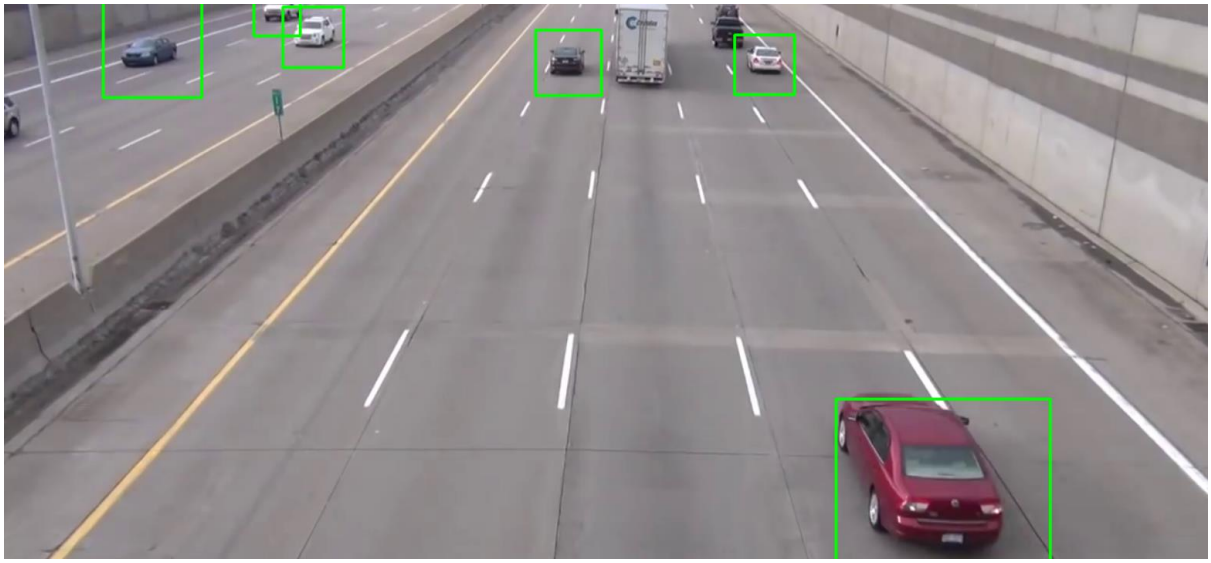
car_cascade.detectMultiScale: Mendeteksi kendaraan dalam frame. Parameter 1.1 dan 3 mengatur skala dan jumlah deteksi minimum.

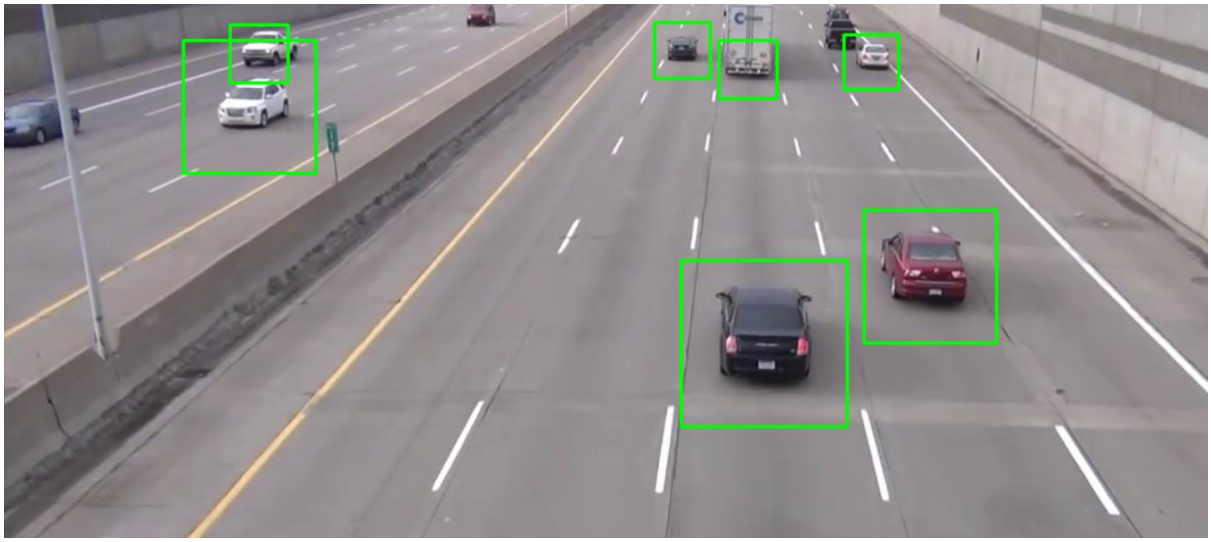
4.2 Hasil dan Data

Telah diperoleh data variasi mobil dari input video/dataset berformat mp4. Hasil tersebut berupa frame hasil deteksi serta prediksi kecepatan setiap mobil.

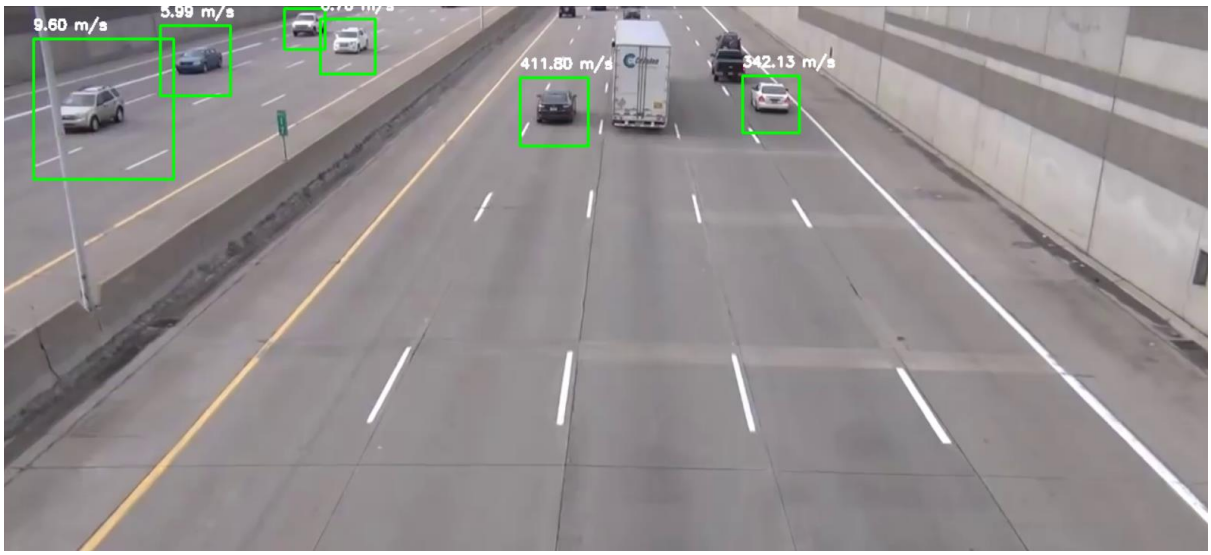
4.1.1 Deteksi Mobil

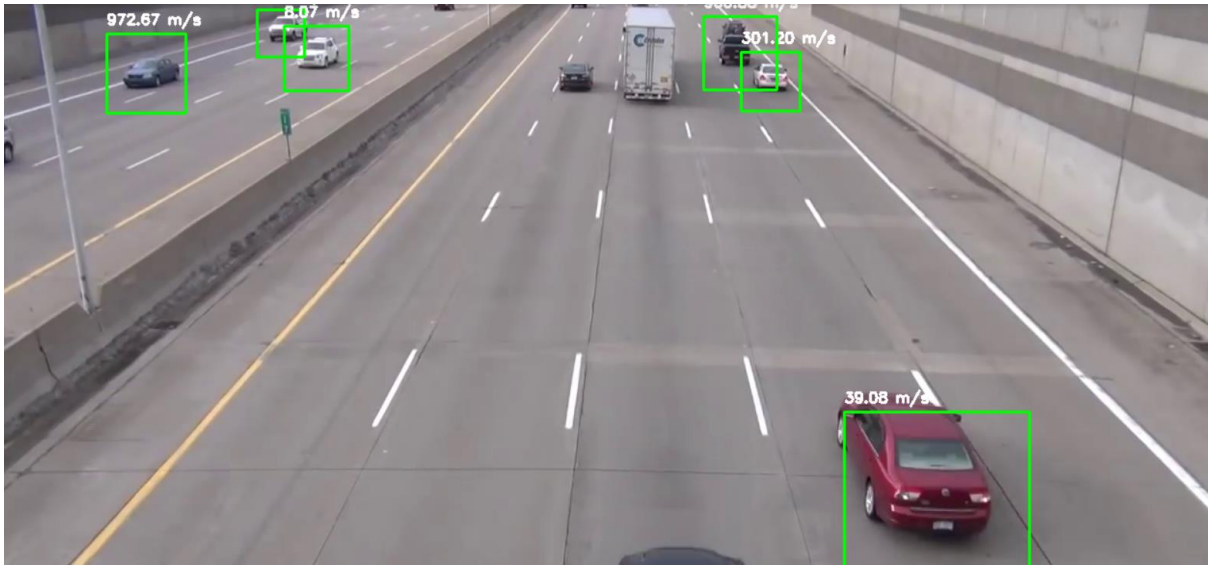


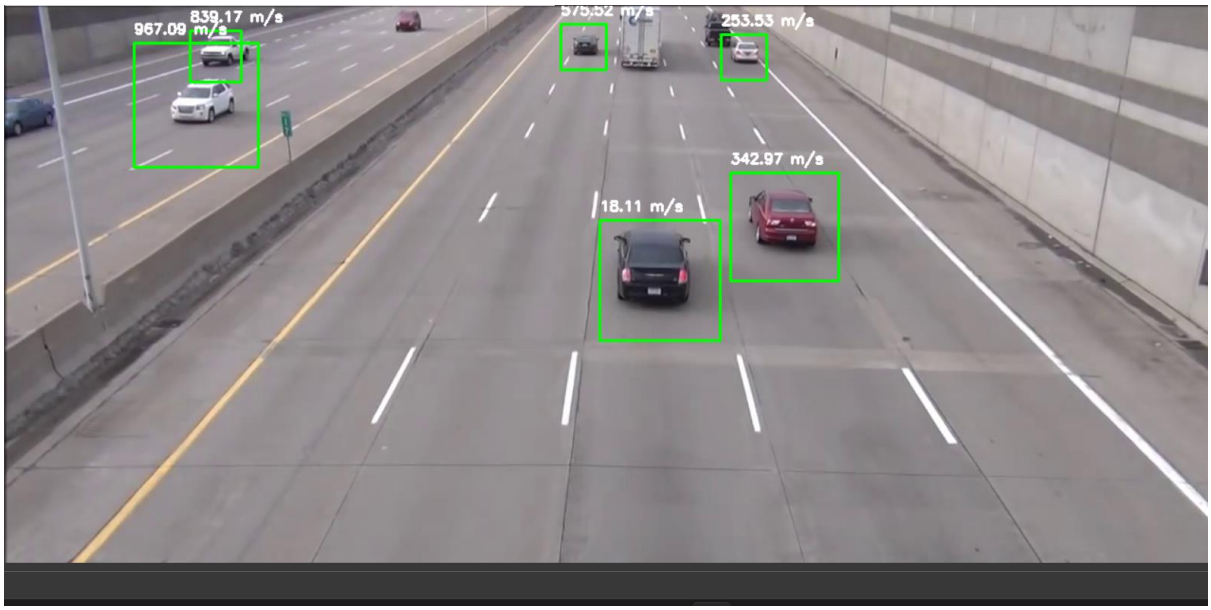




4.1.2 Prediksi Kecepatan





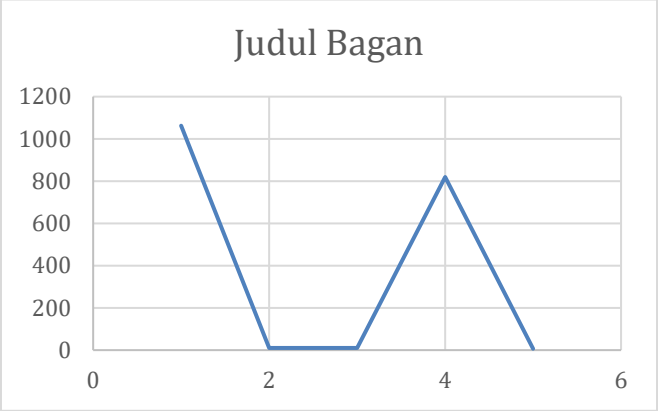


4.1.3 Data dan Grafik

- Variasi Benda I

Variasi Benda 1

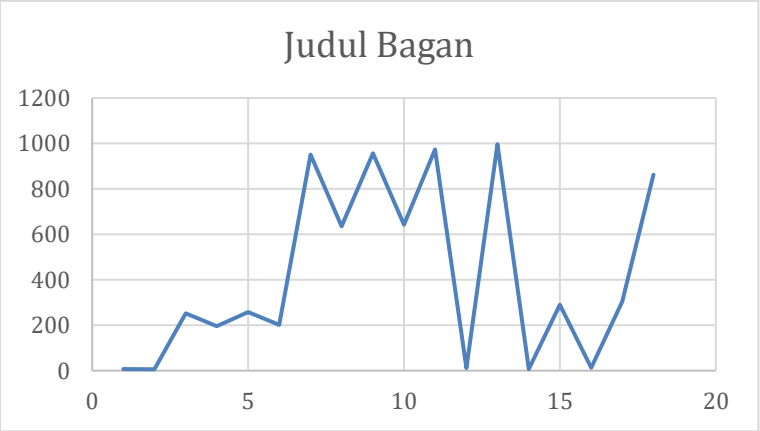
Frame	Kecepatan (m/s)
1	1062.46
2	9.6
3	9.6
4	819.04
5	6.7



- Variasi Benda 2

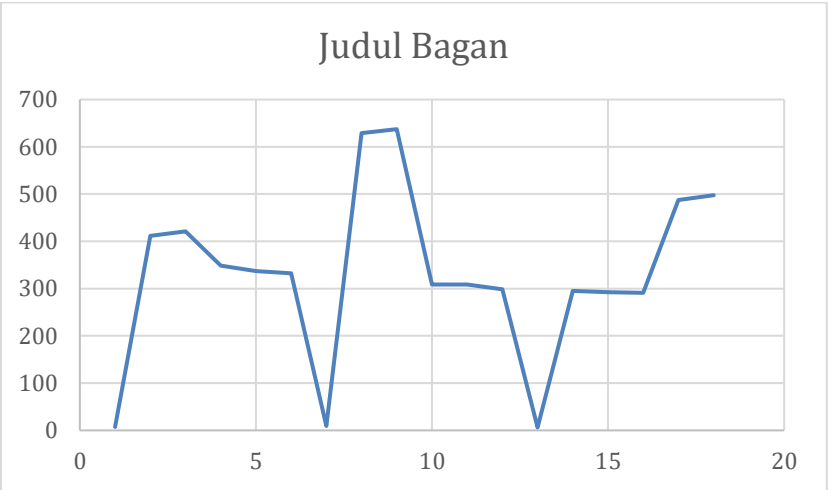
Variasi Benda 2	
Frame	Kecepatan (m/s)
1	7.64
2	5.99
3	251.54
4	195.78

5	257.51
6	201.52
7	950.98
8	634.97
9	956.42
10	641.99
11	972.67
12	10.91
13	996.51
14	7.64
15	290.61
16	12.08
17	304.403
18	861.74



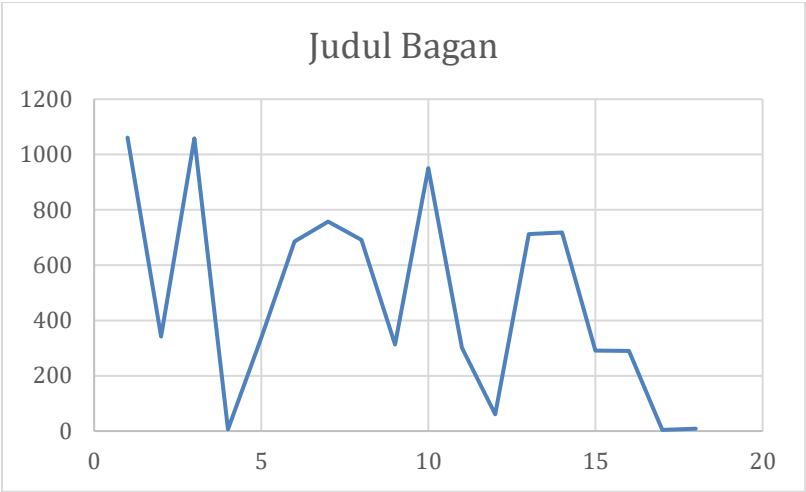
- Variasi Benda 3

Variasi Benda 3	
Frame	Kecepatan (m/s)
1	7.49
2	411.8
3	421.31
4	348.56
5	337.28
6	332.68
7	9.48
8	628.74
9	637.37
10	308.78
11	308.78
12	298.24
13	6.36
14	295.3
15	292.45
16	290.96
17	487.73
18	497.78



- Variasi Benda 4

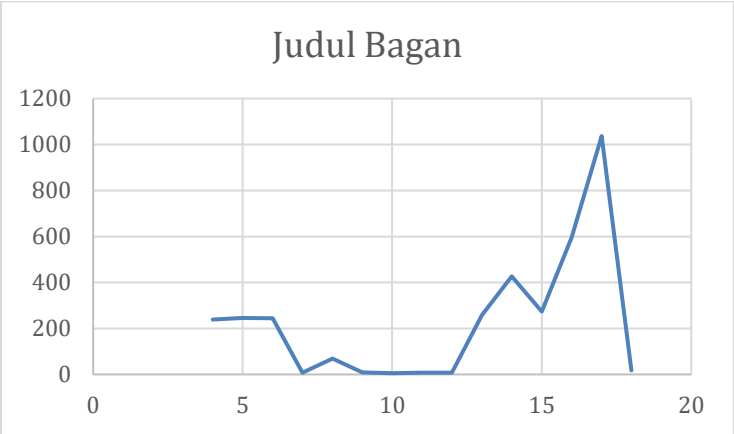
Variasi Benda 4	
Frame	Kecepatan (m/s)
1	1060.94
2	342.13
3	1058.03
4	5.99
5	337.38
6	685.6
7	757.04
8	691.54
9	313.19
10	950.32
11	301.2
12	60.41
13	712.3
14	718.09
15	290.72
16	289.35
17	4.24
18	8.74



- Variasi Benda 5

Variasi Benda 5	
Frame	Kecepatan (m/s)
4	239.18
5	246.32
6	243.8
7	7.49
8	69.42
9	9.12
10	5.4
11	8.07
12	8.07
13	258.48
14	427.39

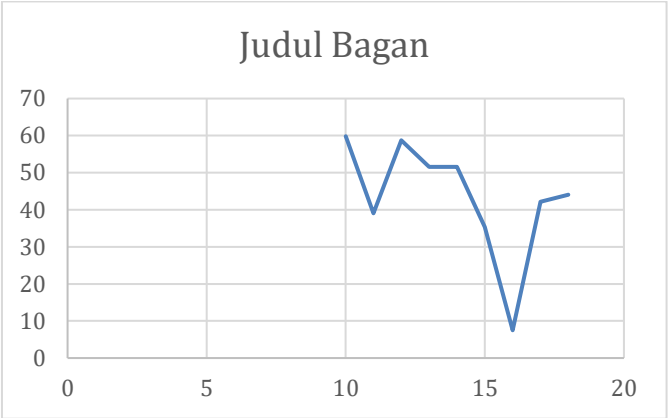
15	273.53
16	594.94
17	1036.89
18	17.54



- Variasi Benda 6

Variasi Benda 6	
Frame	Kecepatan (m/s)
10	59.83
11	39.08
12	58.75
13	51.61
14	51.61

15	35.37
16	7.49
17	42.2
18	44.1



4.2 Pembahasan

Sistem deteksi dan prediksi kecepatan mobil dengan HAAR Cascade Classifier dirancang untuk menganalisis pergerakan mobil dalam video secara real-time. Metode ini menggunakan pembelajaran mesin untuk mendeteksi objek tertentu, seperti mobil, dengan mengidentifikasi pola visual yang sesuai. Dalam video yang dianalisis, setiap mobil yang berhasil terdeteksi akan diberi tanda berupa kotak hijau (bounding box), yang menunjukkan bahwa objek tersebut dikenali sebagai mobil.

Proses ini dilakukan pada setiap frame video, di mana algoritma menganalisis pola visual untuk mendeteksi objek yang sesuai dengan model yang telah dilatih. Setelah mobil terdeteksi, sistem melanjutkan dengan menghitung kecepatan mobil berdasarkan perubahan posisi objek antar frame. Kecepatan dihitung menggunakan rumus fisika sederhana, yaitu membagi jarak perpindahan mobil dengan selang waktu antar frame. Hasil perhitungan kecepatan ini ditampilkan dalam video, memberikan informasi langsung tentang kecepatan mobil dalam satuan seperti meter per detik (m/s).

Program ini memanfaatkan pustaka OpenCV untuk mengolah video, dengan langkah-langkah utama meliputi pembagian video menjadi frame individu, deteksi mobil menggunakan HAAR Cascade Classifier, penghitungan kecepatan berdasarkan perubahan posisi bounding

box, dan visualisasi hasil deteksi serta kecepatan pada video. Sistem ini dirancang untuk bekerja secara efisien dalam kondisi real-time, meskipun ada beberapa keterbatasan, seperti kesulitan dalam mendeteksi objek pada kondisi pencahayaan yang buruk atau saat terdapat noise pada video.

Hasil analisis menunjukkan deteksi dan prediksi kecepatan mobil berdasarkan pemrosesan video menggunakan Haar Cascade Classifier. Metode ini bekerja dengan mendeteksi kendaraan di setiap frame video, lalu menghitung kecepatan berdasarkan perubahan posisi kendaraan dalam interval waktu tertentu. Namun, data yang dihasilkan menunjukkan adanya fluktuasi kecepatan yang signifikan di setiap variasi, bahkan terdapat outlier dengan kecepatan yang sangat tinggi, yang kemungkinan besar merupakan hasil dari kesalahan deteksi.

Pada Variasi Benda 1 terdapat lima frame dengan kecepatan yang tidak konsisten, mulai dari nilai sangat tinggi seperti 1062.46 m/s hingga nilai rendah seperti 6.7 m/s. Fluktuasi ini mengindikasikan kesalahan prediksi, yang bisa disebabkan oleh sudut kamera atau gangguan latar belakang.

Variasi Benda 2 melibatkan 18 frame dengan variasi kecepatan ekstrem, mulai dari 5.99 m/s hingga 996.51 m/s. Beberapa frame menunjukkan outlier yang jauh dari nilai wajar, mencerminkan kelemahan dalam akurasi Haar Cascade, terutama pada perubahan posisi kendaraan yang cepat.

Pada Variasi Benda 3 meskipun terdapat beberapa fluktuasi, data menunjukkan pola yang sedikit lebih stabil dibandingkan variasi lainnya. Namun, kecepatan pada frame tertentu masih menunjukkan nilai yang jauh lebih tinggi dari normal, seperti 637.37 m/s.

Variasi Benda 4 kembali menunjukkan fluktuasi besar dengan kecepatan yang bervariasi dari 4.24 m/s hingga 1060.94 m/s. Prediksi yang tidak wajar ini kemungkinan terjadi karena kendaraan mendekati atau menjauh dari kamera secara cepat, sehingga memengaruhi akurasi deteksi.

Pada Variasi Benda 5 data menunjukkan adanya percepatan signifikan pada frame akhir, dengan kecepatan mencapai 1036.89 m/s. Hal ini menunjukkan kemungkinan deteksi kendaraan lain atau kesalahan segmentasi pada video.

Variasi Benda 6 meskipun lebih sedikit frame, memiliki kecepatan yang relatif konsisten dibandingkan variasi lainnya. Kecepatan berkisar antara 7.49 m/s hingga 59.83 m/s, dengan fluktuasi yang lebih kecil, mencerminkan prediksi yang lebih stabil.

Secara keseluruhan, hasil analisis ini mengungkapkan bahwa Haar Cascade Classifier mampu mendeteksi kendaraan, tetapi tidak cukup akurat untuk prediksi kecepatan yang andal. Fluktuasi dan outlier yang signifikan menunjukkan perlunya metode pelengkap seperti algoritma pelacakan (misalnya Kalman Filter) untuk meningkatkan akurasi. Selain itu, penggunaan kamera dengan resolusi lebih tinggi dan metode deteksi yang lebih canggih seperti YOLO dapat membantu mengatasi masalah ini, sehingga data yang dihasilkan dapat dimanfaatkan untuk aplikasi seperti sistem pemantauan lalu lintas atau analisis pergerakan kendaraan.

BAB V

PENUTUP

5.1 Kesimpulan

Pengembangan sistem berbasis Haar Cascade Classifier berhasil dilakukan untuk mendeteksi kendaraan dalam video. Sistem ini mampu mendeteksi keberadaan kendaraan secara real-time dengan menampilkan bounding box di sekitar objek yang terdeteksi. Namun, akurasi deteksi kendaraan yang dihasilkan oleh sistem sangat bergantung pada kualitas model Haar Cascade yang digunakan, parameter deteksi (seperti skala dan jumlah tetangga), serta kualitas video (pencahayaan, resolusi, dan sudut pandang). Untuk evaluasi lebih lanjut, tingkat akurasi dapat diukur dengan membandingkan hasil deteksi dengan data ground truth, mencatat tingkat kesalahan (false positives dan false negatives), dan menyesuaikan parameter model untuk meningkatkan kinerja.

DAFTAR PUSTAKA

Mahalakshmi, P. D., & Babu, M. (2019). Vehicle Speed Estimation using Haar Classifier Algorithm. *International Journal of Trend in Scientific Research and Development (IJTSRD)*, *4*(1), 243-246.

Tian, M. Z., Liu, L., Lu, J. Y., & Cheng, Y. (2022, July). Vehicle recognition based on Haar features and Adaboost cascade classifier. In *Journal of Physics: Conference Series* (Vol. 2303, No. 1, p. 012052). IOP Publishing.

Siddiqua, A., & Saher, A. (2024). Sumera (2024) Vehicle Speed Detection using Haar Cascade Classifier and Correlation Tracking. *Indian Journal of Science and Technology*, *17*(8), 741-750.

Yulina, S. (2021). Implementation of Haar Cascade Classifier for Face Detection and Grayscale Image Transformation Using OpenCV. *Jurnal Komputer Terapan*, *7*(1), 100-109.

Reinius, S. (2013). Object recognition using the OpenCV Haar cascade-classifier on the iOS platform.